Requirements and where they are met

Every requirement from the thesis assignment, the committee's 50 % review and the supervisor's feedback, with its status, the measured result and the code that implements it

Project: Latency and Accuracy Optimization for OCR to LLM Document Processing Pipelines · 20 October 2026 · Benchmark: `outputs/2026-10-18_23-52-41_full_benchmark`

27	1	4	0
requirements done in code and measured	done in a different way than suggested, with reason	need the author's writing (material provided)	experiments still running

How to read this

- **Done**: implemented, run on the 600 invoices, and the result is stated. **Done – alternative**: met in a different way than suggested; the reason is given. **Author's writing**: part of the thesis text, which the author writes; the material it needs is listed.
- Code excerpts are copied from the files when this document was built; the file and line numbers are shown above each excerpt. Numbers come from the benchmark output folders named in each entry.

1. Overview

ID	Source	Requirement	Status	Where
G1	Thesis assignment	Design and compare two OCR→LLM architectures – a simple sequential pipeline and an optimised asynchronous pipeline …	**Done**	Architecture_and_Benchmark_Report.pdf, sections 1 and 6
T1	Thesis assignment	Analyse existing OCR→LLM pipelines and identify key performance issues such as latency, cost and scalability.	**Author's writing**	`pipeline/arch_a.py:29`
T2	Thesis assignment	Review relevant OCR tools, caching methods, compression techniques and asynchronous processing approaches.	**Author's writing**	Literature_Sources.pdf; Thesis_Outline.pdf, chapter 1
T3	Thesis assignment	Develop two pipeline prototypes – A: baseline sequential OCR→Embeddings→LLM; B: optimised asynchronous pipeline with …	**Done**	`pipeline/arch_a.py:29` `pipeline/arch_b.py:154` `pipeline/arch_b.py:175`
T4	Thesis assignment	Describe both architectures, outlining modules, data flow and the integration of caching and compression.	**Done**	`pipeline/arch_b.py:77` `pipeline/arch_b.py:87`
T5	Thesis assignment	Evaluate both pipelines on ~600 real and synthetic documents using accuracy (F1), latency, throughput, memory and cost.	**Done**	`evaluation/scorer.py:87` `pipeline/metrics.py:123` `pipeline/metrics.py:132`
T6	Thesis assignment	Compare and analyse results, discuss trade-offs, and propose improvements for future AI-based document processing …	**Done**	Architecture_and_Benchmark_Report.pdf, sections 6–8
C1	Committee, 50 % review	Discuss modern vision-language models (VLMs) that read documents directly, and justify scientifically why a …	**Done**	`evaluation/vlm_baseline.py:49`
C2	Committee, 50 % review	Identify the bugs that led to the incorrect results presented to the committee.	**Done**	`pipeline/llm.py:10` `pipeline/dataset.py:45` `evaluation/scorer.py:45`
C3	Committee, 50 % review	Fix the code and completely re-run the evaluations on the scaled dataset.	**Done**	`evaluation/run_experiments.py:218`
C4	Committee, 50 % review	Write a proper analytical part (IACS A-3, A-4): a literature review of other authors' scientific work, existing OCR …	**Author's writing**	Literature_Sources.pdf; Thesis_Outline.pdf
C5	Committee, 50 % review	Restructure the thesis: analytical part first, then system design, then dataset and evaluation method, then results.	**Author's writing**	Thesis_Outline.pdf
S1	Supervisor, 25 % feedback	Make the routing logic explicit: a complexity scorer on token count and OCR confidence (IF token_count < 500 AND …	**Done**	`pipeline/routing.py:26` `pipeline/config.py:44`
S2	Supervisor, 25 % feedback	OCR and LLM inference are blocking: use loop.run_in_executor with a ProcessPoolExecutor so the event loop stays free.	**Done**	`pipeline/arch_b.py:147` `pipeline/arch_b.py:154`
S3	Supervisor, 25 % feedback	Name specific quantization tools – bitsandbytes (4-bit loading) or llama.cpp (GGUF) – instead of 'quantization' in …	**Done**	`pipeline/llm.py:18`
S4	Supervisor, 25 % feedback	Robust F1: use Levenshtein distance or fuzzy-matching thresholds so minor OCR artefacts ('l' vs '1') do not count as …	**Done**	`evaluation/scorer.py:14` `evaluation/scorer.py:23`
P1	Supervisor, plan for the 50 % submission	Implement the core of Architecture B (asyncio and SQLite caching) alongside Architecture A.	**Done**	`pipeline/cache.py:21` `pipeline/arch_b.py:45`

ID	Source	Requirement	Status	Where
P2	Supervisor, plan for the 50 % submission	Hardware-aware model selection for the RTX 3050: adopt quantization early and test 4-bit models (bitsandbytes or GGUF).	**Done**	see S3, M3
P3	Supervisor, plan for the 50 % submission	Measure the OCR confidence score and use it to define the routing threshold mathematically.	**Done**	`pipeline/ocr.py:109`
P4	Supervisor, plan for the 50 % submission	Describe the data synthesis or acquisition strategy for reaching ~600 documents.	**Done**	`generate_dataset.py:128`
P5	Supervisor, plan for the 50 % submission	Compare Tesseract and EasyOCR, measuring accuracy and latency.	**Done**	`pipeline/ocr.py:130` `evaluation/ocr_comparison.py:16`
P6	Supervisor, plan for the 50 % submission	Pilot data: 15 documents with JSON ground truth; derive routing thresholds from this data.	**Done**	see M4
B1	Supervisor, before the 50 % submission	Architecture A must be a sequential LLM pipeline (OCR→Embeddings→LLM) as planned, not regular expressions.	**Done**	see T3
B2	Supervisor, before the 50 % submission	Scale the pilot to 10–15 documents and outline the strategy for ~600 documents.	**Done**	see P4
B3	Supervisor, before the 50 % submission	Explain explicitly how FLAN-T5 is quantized, naming the tools (bitsandbytes or GGUF).	**Done**	see S3
M1	Supervisor, 75 % feedback	Generate ground truth automatically for the synthetic invoices; for real invoices consider a teacher model (e.g. …	**Done – alternative**	`generate_dataset.py:95` `evaluation/relabel_totals.py:33`
M2	Supervisor, 75 % feedback	Use Levenshtein distance or a fuzzy-matching threshold (e.g. ≥ 90 % similarity) in the evaluation script.	**Done**	see S4
M3	Supervisor, 75 % feedback	Document the use of bitsandbytes and test whether 4-bit quantization reduces latency and memory on the RTX 3050 …	**Done**	see S3
M4	Supervisor, 75 % feedback	Use the 600-document results to statistically finalise the routing thresholds (exact token count and OCR confidence).	**Done**	`evaluation/routing_analysis.py:14` `evaluation/routing_analysis.py:75`
E1	Supervisor, e-mails	Three runs per configuration.	**Done**	see C3
E2	Supervisor, e-mails	Report cost in € per 100 documents.	**Done**	`pipeline/metrics.py:123` `pipeline/metrics.py:144`
E3	Supervisor, e-mails	Document the hardware (RTX 3050).	**Done**	`evaluation/run_experiments.py:54`

2. Evidence per requirement

G1 · Thesis assignment

Asked for	*Design and compare two OCR→LLM architectures – a simple sequential pipeline and an optimised asynchronous pipeline with caching and model compression – to improve latency, throughput and cost-efficiency while keeping high accuracy.*
Status	**Done**
What was done	Both architectures are implemented and compared on all 600 invoices: 12 configurations, 3 runs each, plus a latency experiment with invoices arriving one at a time.
Result	Throughput 53.1 → 301.7 invoices/min (5.7×); cost per 100 invoices €0.0110 → €0.0019; compute per invoice 1.12 → 0.72 s; F1 0.9406 in both. Median latency with invoices arriving one at a time – 30/min: A 1.7 s, B 7.9 s, B-greedy 1.0 s; 48/min: A 3.9 s, B 5.3 s, B-greedy 1.0 s; 120/min: A 22.8 s, B 3.0 s, B-greedy 1.2 s; 240/min: A 29.9 s, B 2.8 s, B-greedy 1.8 s (outputs/2026-10-20_01-01-33_latency_60docs).
Documented in	Architecture_and_Benchmark_Report.pdf, sections 1 and 6

T1 · Thesis assignment

Asked for	*Analyse existing OCR→LLM pipelines and identify key performance issues such as latency, cost and scalability.*
Status	**Author's writing**
What was done	The measured part is done: the baseline's limits are quantified and B's scaling limit is explained by a stage model. Comparing these with other authors' pipelines is the literature part of the thesis; sources are collected in Literature_Sources.pdf.
Result	In A the GPU is idle 53 % of the time (while OCR runs) and 1 of 12 CPU threads works. B's wall time equals the slower of its two stages (OCR pool vs GPU thread) within 8 % for every variant.
Documented in	Architecture_and_Benchmark_Report.pdf, sections 3 and 6.4

pipeline/arch_a.py · lines 29–42

```
29      def process(self, doc: Document) -> DocumentRecord:
30          truth = doc.load_truth()
31          started = time.perf_counter()
32          try:
33              ocr = run_ocr(doc.image_path, OCR_LAYOUT)
34              mark = time.perf_counter()
35              context = self.selector.select(ocr.text)
36              embed_time = time.perf_counter() - mark
37              mark = time.perf_counter()
38              prediction = self.extractor.extract_sequential(context, MAX_NEW_TOKENS_FULL)
39              llm_time = time.perf_counter() - mark
40              prediction, total_source = finalize(prediction, ocr.text, RECONCILE_TOTALS)
41              latency = time.perf_counter() - started
42              features = compute_features(
```

T2 · Thesis assignment

Asked for	*Review relevant OCR tools, caching methods, compression techniques and asynchronous processing approaches.*
Status	**Author's writing**
What was done	Each technology now has a measured comparison on the thesis hardware that the review can cite; the review of the literature itself is the author's.
Result	OCR tools: see P5. Compression: fp32 / fp16 / 8-bit / 4-bit (see M3). Caching: cold vs warm (see P1). Asynchronous processing: 1 / 3 / 6 OCR workers and batching on/off (see S2).
Documented in	Literature_Sources.pdf; Thesis_Outline.pdf, chapter 1

T3 · Thesis assignment

Asked for	*Develop two pipeline prototypes – A: baseline sequential OCR→Embeddings→LLM; B: optimised asynchronous pipeline with caching, batching and quantized models.*
Status	**Done**
What was done	A runs OCR, the MiniLM embedding filter and Flan-T5 one invoice at a time. B adds the cache lookup, a process pool for OCR, GPU batches of 8 and routing between a 4-bit and an fp32 model.
Documented in	docs/ARCHITECTURE.md, sections 3 and 4

pipeline/arch_a.py · lines 29–42

```
29      def process(self, doc: Document) -> DocumentRecord:
30          truth = doc.load_truth()
31          started = time.perf_counter()
32          try:
33              ocr = run_ocr(doc.image_path, OCR_LAYOUT)
34              mark = time.perf_counter()
35              context = self.selector.select(ocr.text)
36              embed_time = time.perf_counter() - mark
37              mark = time.perf_counter()
38              prediction = self.extractor.extract_sequential(context, MAX_NEW_TOKENS_FULL)
39              llm_time = time.perf_counter() - mark
40              prediction, total_source = finalize(prediction, ocr.text, RECONCILE_TOTALS)
41              latency = time.perf_counter() - started
42              features = compute_features(
```

pipeline/arch_b.py · lines 154–159

```
154         gpu_executor = ThreadPoolExecutor(max_workers=1)
155         with ProcessPoolExecutor(max_workers=self.config.ocr_workers, initializer=configure_worker) as pool:
156             tasks = [asyncio.ensure_future(ocr_job(doc, pool)) for doc in pending]
157             batch: list[tuple[Document, OcrResult, dict, float]] = []
158             gpu_jobs = []
159             for completed in asyncio.as_completed(tasks):
```

pipeline/arch_b.py · lines 175–180

```
175                 batch.append((doc, result, truths[doc.index], submitted))
176                 if len(batch) >= self.config.batch_size:
177                     gpu_jobs.append(loop.run_in_executor(gpu_executor, self._process_batch, batch))
178                     batch = []
179             if batch:
180                 gpu_jobs.append(loop.run_in_executor(gpu_executor, self._process_batch, batch))
```

T4 · Thesis assignment

Asked for	*Describe both architectures, outlining modules, data flow and the integration of caching and compression.*
Status	**Done**
What was done	Both architectures are described module by module with diagrams, data flow, a to-scale schedule and the reason for each part, including where the cache and the quantized model sit.
Documented in	Architecture_and_Benchmark_Report.pdf, sections 2–4; docs/ARCHITECTURE.md

pipeline/arch_b.py · lines 77–79

```
77      def _process_batch(self, batch: list[tuple[Document, OcrResult, dict, float]]) -> list[DocumentRecord]:
78          texts = [ocr.text for _, ocr, _, _ in batch]
79          mark = time.perf_counter()
```

pipeline/arch_b.py · lines 87–98

```
87          routes = [decide_route(feature, self.config.thresholds, self.config.routing) for feature in features]
88          predictions: list[dict | None] = [None] * len(batch)
89          llm_each: list[float] = [0.0] * len(batch)
90          for route in (FAST, FULL):
91              positions = [i for i, r in enumerate(routes) if r == route]
92              if not positions:
93                  continue
94              mark = time.perf_counter()
95              outputs = self.extractors[route].extract_batch([contexts[i] for i in positions],
MAX_NEW_TOKENS[route])
96              elapsed = (time.perf_counter() - mark) / len(positions)
97              for position, output in zip(positions, outputs):
98                  predictions[position] = output
```

T5 · Thesis assignment

Asked for	*Evaluate both pipelines on ~600 real and synthetic documents using accuracy (F1), latency, throughput, memory and cost.*
Status	**Done**
What was done	600 invoices (300 real, 300 synthetic). F1 per field and overall; wall-clock throughput; service time and latency; RAM of the process tree and GPU memory sampled 4× per second; cost per 100 invoices.
Result	A: 677.9 ± 7.4 s, B: 119.4 ± 4.7 s for 600 invoices; GPU peak 1067 vs 1996 MB; RAM 1438 vs 2012 MB.
Documented in	outputs/2026-10-18_23-52-41_full_benchmark/summary.md

evaluation/scorer.py · lines 87–92

```
87  def _prf(counts: dict) -> dict:
88      tp, fp, fn, tn = counts["TP"], counts["FP"], counts["FN"], counts["TN"]
89      precision = tp / (tp + fp) if tp + fp else 0.0
90      recall = tp / (tp + fn) if tp + fn else 0.0
91      f1 = 2 * precision * recall / (precision + recall) if precision + recall else 0.0
92      accuracy = (tp + tn) / (tp + tn + max(fp, fn)) if tp + tn + max(fp, fn) else 0.0
```

pipeline/metrics.py · lines 123–124

```
123     total_hours = wall_time / 3600
124     total_cost = total_hours * GPU_HOURLY_COST_EUR
```

pipeline/metrics.py · lines 132–132

```
132         "throughput_docs_per_min": round(docs / wall_time * 60, 3) if wall_time else 0.0,
```

pipeline/metrics.py · lines 87–91

```
87      def _loop(self) -> None:
88          while not self._stop.is_set():
89              self.rss_samples.append(process_tree_rss_mb())
90              self.gpu_samples.append(gpu_memory_used_mb())
91              self._stop.wait(self.interval)
```

T6 · Thesis assignment

Asked for	*Compare and analyse results, discuss trade-offs, and propose improvements for future AI-based document processing systems.*
Status	**Done**
What was done	The technical comparison, the trade-offs (speed vs memory, throughput vs latency, quantization) and measured improvement options are in the report. The discussion chapter is the author's.
Documented in	Architecture_and_Benchmark_Report.pdf, sections 6–8

C1 · Committee, 50 % review

Asked for	*Discuss modern vision-language models (VLMs) that read documents directly, and justify scientifically why a dedicated OCR→LLM pipeline is still necessary or advantageous (hardware, cost, control).*
Status	**Done**
What was done	A VLM was measured on the same GPU, invoices and scorer, so the argument rests on numbers from the thesis hardware. Sources on VLMs vs OCR are in Literature_Sources.pdf. The argument itself is written by the author.
Result	Qwen2-VL-2B (4-bit NF4) reading the image directly: F1 0.966 (real 0.930), 3.90 s per invoice, GPU peak 1745 MB, at most 1024 visual tokens – against A 1.12 s and B 0.20 s of wall time per invoice, F1 0.941 (outputs/2026-10-20_23-54-32_vlm_baseline).
Documented in	Architecture_and_Benchmark_Report.pdf, section 7; Literature_Sources.pdf

evaluation/vlm_baseline.py · lines 49–55

```
49          config = BitsAndBytesConfig(
50              load_in_4bit=True,
51              bnb_4bit_quant_type="nf4",
52              bnb_4bit_use_double_quant=True,
53              bnb_4bit_compute_dtype=torch.bfloat16,
54          )
55          self.model = AutoModelForImageTextToText.from_pretrained(VLM_NAME, quantization_config=config,
device_map=device).eval()
```

C2 · Committee, 50 % review

Asked for	*Identify the bugs that led to the incorrect results presented to the committee.*
Status	**Done**
What was done	Eleven defects were found and fixed; the most consequential: Flan-T5 was asked for JSON, which it cannot write (no { } tokens), so every answer came from a regex; the scorer scored the wrong object; amounts were compared on their first digits; A and B used different scorers and thresholds. A twelfth, in the date parser, was found during the OCR comparison: dates written without a space after the comma ('April 19,2024') were read with the current year. It was fixed with a unit test and every stored run was re-scored; the main benchmark did not change.
Result	With the defects fixed, F1 is 0.941 for both architectures on 600 invoices.
Documented in	docs/CHANGES.pdf; Architecture_Report.pdf, section 3.1

pipeline/llm.py · lines 10–11

```
10  def build_prompt(context: str, field: str) -> str:
11      return f"{context}\n\nQuestion: {QUESTIONS[field]}\nAnswer:"
```

pipeline/dataset.py · lines 45–45

```
45      documents.sort(key=lambda doc: doc.index)
```

evaluation/scorer.py · lines 45–53

```
45  def match_total(pred, truth) -> bool:
46      fa, fb = parse_amount(pred), parse_amount(truth)
47      if fa is not None and fb is not None:
48          return abs(fa - fb) < 0.005
49      return fuzzy_equal(digits_only(pred), digits_only(truth))
50
51
52  MATCHERS = {"invoice_number": match_invoice_number, "date": match_date, "total": match_total}
53
```

pipeline/normalize.py · lines 35–40

```
35 def parse_date(value) -> date | None:
36     if is_null(value):
37         return None
38     text = clean_text(value)
39     text = re.sub(r",(?=\d)", ", ", text)
40     text = LETTER_DIGIT_BOUNDARY.sub(" ", text)
```

C3 · Committee, 50 % review

Asked for	*Fix the code and completely re-run the evaluations on the scaled dataset.*
Status	**Done**
What was done	All evaluations were re-run with the corrected code on all 600 invoices. Runs disturbed by other programs were detected by their OCR time per invoice and repeated; the disturbed runs are kept in discarded/.
Result	12 configurations × 3 runs = 36 clean runs of 600 invoices.
Documented in	outputs/2026-10-18_23-52-41_full_benchmark/run_info.json

evaluation/run_experiments.py · lines 218–218

```
218     parser.add_argument("--runs", type=int, default=3)
```

C4 · Committee, 50 % review

Asked for	*Write a proper analytical part (IACS A-3, A-4): a literature review of other authors' scientific work, existing OCR solutions and their underperformance; do not claim to be the pioneer.*
Status	**Author's writing**
What was done	Support provided: a verified source list with summaries (Literature_Sources.pdf), the chapter plan (Thesis_Outline.pdf, chapter 1) and measured evidence for the technology choices (P5, M3, C1).
Documented in	Literature_Sources.pdf; Thesis_Outline.pdf

C5 · Committee, 50 % review

Asked for	*Restructure the thesis: analytical part first, then system design, then dataset and evaluation method, then results.*
Status	**Author's writing**
What was done	The chapter plan in Thesis_Outline.pdf follows exactly this order; the technical report follows it too (design → method → results).
Documented in	Thesis_Outline.pdf

S1 · Supervisor, 25 % feedback

Asked for	*Make the routing logic explicit: a complexity scorer on token count and OCR confidence (IF token_count < 500 AND ocr_confidence > 0.9 THEN quantized model ELSE full model).*
Status	**Done**
What was done	The router computes the token count of the OCR text and the mean OCR word confidence and applies exactly this rule; the thresholds are parameters.
Result	Initial thresholds 300 tokens / confidence 85 send 505 invoices to the 4-bit model and 95 to fp32; the data-derived thresholds (M4) are 525 / 70.

pipeline/routing.py · lines 26–33

```
26 def decide_route(features: RouteFeatures, thresholds: RoutingThresholds, policy: str = "auto") -> str:
27     if policy == "all_fast":
28         return FAST
29     if policy == "all_full":
30         return FULL
31     if features.token_count <= thresholds.max_tokens_fast and features.confidence >=
thresholds.min_confidence_fast:
32         return FAST
33     return FULL
```

pipeline/config.py · lines 44–46

```
44 class RoutingThresholds:
45     max_tokens_fast: int = 300
46     min_confidence_fast: float = 85.0
```

S2 · Supervisor, 25 % feedback

Asked for	*OCR and LLM inference are blocking: use loop.run_in_executor with a ProcessPoolExecutor so the event loop stays free.*
Status	**Done**
What was done	OCR runs in a ProcessPoolExecutor and GPU work in a single-thread executor, both through loop.run_in_executor; the event loop only schedules.
Result	Parallel OCR is worth 2.8× (1 vs 3 workers); GPU batching 2.1×.

pipeline/arch_b.py · lines 147–150

```
147         async def ocr_job(doc: Document, pool):
148             try:
149                 result = await loop.run_in_executor(pool, run_ocr, doc.image_path, OCR_LAYOUT)
150                 return doc, result, None
```

pipeline/arch_b.py · lines 154–157

```
154         gpu_executor = ThreadPoolExecutor(max_workers=1)
155         with ProcessPoolExecutor(max_workers=self.config.ocr_workers, initializer=configure_worker) as pool:
156             tasks = [asyncio.ensure_future(ocr_job(doc, pool)) for doc in pending]
157             batch: list[tuple[Document, OcrResult, dict, float]] = []
```

S3 · Supervisor, 25 % feedback

Asked for	*Name specific quantization tools – bitsandbytes (4-bit loading) or llama.cpp (GGUF) – instead of 'quantization' in general.*
Status	**Done**
What was done	bitsandbytes is used: 4-bit NF4 with double quantization (QLoRA-style) and 8-bit LLM.int8, loaded through transformers' BitsAndBytesConfig. GGUF/llama.cpp, the alternative the supervisor named, was not needed: bitsandbytes runs inside the same PyTorch pipeline on the GPU.
Documented in	docs/ARCHITECTURE.md, section 2.3

pipeline/llm.py · lines 18–35

```
18  def load_model(precision: str, device: str = "cuda"):
19      if precision not in PRECISIONS:
20          raise ValueError(f"Unknown precision {precision}; expected one of {PRECISIONS}")
21      if precision == "8bit":
22          config = BitsAndBytesConfig(load_in_8bit=True)
23          return AutoModelForSeq2SeqLM.from_pretrained(LLM_NAME, quantization_config=config,
device_map=device).eval()
24      if precision == "4bit":
25          config = BitsAndBytesConfig(
26              load_in_4bit=True,
27              bnb_4bit_quant_type="nf4",
28              bnb_4bit_use_double_quant=True,
29              bnb_4bit_compute_dtype=torch.float16,
30          )
31          return AutoModelForSeq2SeqLM.from_pretrained(LLM_NAME, quantization_config=config,
device_map=device).eval()
32      dtype = {"fp32": torch.float32, "fp16": torch.float16, "bf16": torch.bfloat16}[precision]
33      model = AutoModelForSeq2SeqLM.from_pretrained(LLM_NAME, torch_dtype=dtype)
34      return model.to(device).eval()
35
```

S4 · Supervisor, 25 % feedback

Asked for	*Robust F1: use Levenshtein distance or fuzzy-matching thresholds so minor OCR artefacts ('l' vs '1') do not count as errors.*
Status	**Done**
What was done	Invoice numbers match when RapidFuzz ratio (normalised Levenshtein similarity) ≥ 90 on the canonical form, dates when they denote the same day, totals when they are the same amount.
Documented in	docs/EVALUATION.md, section 2

evaluation/scorer.py · lines 14–20

```
14  FUZZY_THRESHOLD = 90.0
15
16
17  def fuzzy_equal(a: str, b: str) -> bool:
18      if not a or not b:
19          return False
20      return fuzz.ratio(a, b) >= FUZZY_THRESHOLD
```

evaluation/scorer.py · lines 23–32

```
def match_invoice_number(pred, truth) -> bool:
    a = normalize_invoice_number(pred)
    b = normalize_invoice_number(truth)
    if a == b:
        return True
    da, db = digits_only(a), digits_only(b)
    if da and db and da == db and len(db) >= 3:
        return True
    return fuzzy_equal(a, b)

```

P1 · Supervisor, plan for the 50 % submission

Asked for	*Implement the core of Architecture B (asyncio and SQLite caching) alongside Architecture A.*
Status	**Done**
What was done	B looks every invoice up in a SQLite cache keyed by the SHA-256 of the image bytes plus the model configuration before any computation.
Result	Cold run 119.4 s; the same 600 invoices again 0.59 s (600 cache hits); without the cache 125.9 s, so the lookup costs nothing measurable.

pipeline/cache.py · lines 21–27

```
    def make_key(image_path: Path, namespace: str) -> str:
        digest = hashlib.sha256()
        digest.update(namespace.encode("utf-8"))
        with open(image_path, "rb") as handle:
            for chunk in iter(lambda: handle.read(1 << 20), b""):
                digest.update(chunk)
        return digest.hexdigest()
```

pipeline/arch_b.py · lines 45–52

```
    def _lookup(self, doc: Document, truth: dict) -> DocumentRecord | None:
        if self.cache is None:
            return None
        started = time.perf_counter()
        key = ResultCache.make_key(doc.image_path, self.namespace)
        payload = self.cache.get(key)
        if payload is None:
            return None
```

P2 · Supervisor, plan for the 50 % submission

Asked for	*Hardware-aware model selection for the RTX 3050: adopt quantization early and test 4-bit models (bitsandbytes or GGUF).*
Status	**Done**
What was done	Flan-T5-base (248 M parameters) was chosen so that fp32 and 4-bit copies fit together in 4 GB; every precision was measured.
Result	Pipeline GPU peak: fp32 1645 MB, 4-bit 1102 MB, fp16 1049 MB, 8-bit 1176 MB.
See also	S3, M3

P3 · Supervisor, plan for the 50 % submission

Asked for	*Measure the OCR confidence score and use it to define the routing threshold mathematically.*
Status	**Done**
What was done	Tesseract's image_to_data returns a confidence per word; the mean is a routing feature. The threshold is chosen by a grid search over token count and confidence (M4).
See also	M4

pipeline/ocr.py · lines 109–113

```
109 def run_ocr_data(image_path) -> dict:
110     if pytesseract.pytesseract.tesseract_cmd != TESSERACT_CMD:
111         configure_worker()
112     image = _load_image(image_path)
113     return pytesseract.image_to_data(image, output_type=pytesseract.Output.DICT)
```

P4 · Supervisor, plan for the 50 % submission

Asked for	*Describe the data synthesis or acquisition strategy for reaching ~600 documents.*
Status	**Done**
What was done	300 synthetic invoices are generated by a script that writes each image and its ground-truth JSON together; 300 real invoices (public templates and scans) were labelled by hand.
Documented in	docs/EVALUATION.md, section 1
See also	M1

generate_dataset.py · lines 128–129

```
128         create_invoice_image(data, img_path)
129         save_json(data, json_path)
```

P5 · Supervisor, plan for the 50 % submission

Asked for	*Compare Tesseract and EasyOCR, measuring accuracy and latency.*
Status	**Done**
What was done	Both engines read all 600 invoices; each engine's text goes through the identical Architecture A downstream and scorer, so the F1 difference is the OCR engine's. EasyOCR uses the same row-by-row text layout as Tesseract.
Result	Tesseract: 0.89 s per real invoice on the CPU, no GPU memory, F1 0.941 (real 0.892). EasyOCR: 8.00 s per real invoice on the GPU with a 3883 MB peak, F1 0.934 (real 0.871). Same downstream and scorer for both (outputs/2026-10-20_01-52-39_ocr_comparison).
Documented in	Architecture_and_Benchmark_Report.pdf, section 6.12

pipeline/ocr.py · lines 130–141

```
130 class EasyOcrEngine:
131     def __init__(self, gpu: bool = True):
132         import easyocr
133 
134         self.reader = easyocr.Reader(["en"], gpu=gpu, verbose=False)
135 
136     def run(self, image_path, layout: str = "rows") -> OcrResult:
137         import numpy
138 
139         start = time.perf_counter()
140         image = numpy.array(_load_image(image_path))
141         segments: list[dict] = []
```

evaluation/ocr_comparison.py · lines 16–31

```
16 def field_found(field: str, truth, text: str) -> bool | None:
17     from rapidfuzz import fuzz
18
19     from pipeline.normalize import clean_text, is_null, normalize_invoice_number, parse_amount
20     from pipeline.validate import _amounts
21
22     if is_null(truth):
23         return None
24     if field == "invoice_number":
25         key = normalize_invoice_number(truth)
26         return bool(key) and key in re.sub(r"[^A-Za-z0-9]", "", text).upper()
27     if field == "total":
28         value = parse_amount(truth)
29         if value is None:
30             return False
31         return any(abs(found - value) < 0.005 for line in text.splitlines() for found, _ in _amounts(line))
```

P6 · Supervisor, plan for the 50 % submission

Asked for	*Pilot data: 15 documents with JSON ground truth; derive routing thresholds from this data.*
Status	**Done**
What was done	Exceeded: the thresholds are derived from all 600 invoices rather than a 15-document pilot.
See also	M4

B1 · Supervisor, before the 50 % submission

Asked for	*Architecture A must be a sequential LLM pipeline (OCR→Embeddings→LLM) as planned, not regular expressions.*
Status	**Done**
What was done	A runs OCR, the MiniLM embedding filter and three Flan-T5 calls in sequence; no regular expression produces an answer.
See also	T3

B2 · Supervisor, before the 50 % submission

Asked for	*Scale the pilot to 10–15 documents and outline the strategy for ~600 documents.*
Status	**Done**
What was done	All 600 invoices are used in every experiment.
See also	P4

B3 · Supervisor, before the 50 % submission

Asked for	*Explain explicitly how FLAN-T5 is quantized, naming the tools (bitsandbytes or GGUF).*
Status	**Done**
What was done	Documented with the exact BitsAndBytesConfig settings for 4-bit NF4 and 8-bit.
Documented in	docs/ARCHITECTURE.md, section 2.3
See also	S3

M1 · Supervisor, 75 % feedback

Asked for	*Generate ground truth automatically for the synthetic invoices; for real invoices consider a teacher model (e.g. GPT-4) to pre-annotate, then verify by hand.*
Status	**Done – alternative**
What was done	Synthetic: the generator writes the JSON together with the image. Real: the existing hand labels were kept and checked arithmetically – 26 totals that held the net instead of the gross amount were corrected only where net + VAT = gross on the OCR text. A teacher model was not needed because labels already existed; the arithmetic check is deterministic and auditable.
Result	26 corrected labels, originals kept in ground_truth_backup_20261018/.

generate_dataset.py · lines 95–104

```
95  def save_json(data, path):
96
97      gt = {
98          "invoice_number": data["invoice_number"],
99          "date": data["date"],
100         "total": data["total"]
101     }
102
103     with open(path, "w") as f:
104         json.dump(gt, f, indent=4)
```

evaluation/relabel_totals.py · lines 33–44

```
33  def find_totals(text: str):
34      if "gross" not in text.lower():
35          return None
36      tail = amounts_in(text)[-24:]
37      values = [v for v, _ in tail]
38      if not values:
39          return None
40      gross = values[-1]
41      for i, net in enumerate(values):
42          for vat in values[i + 1 :]:
43              if abs(net + vat - gross) < TOLERANCE and net > vat:
44                  return net, vat, gross
```

M2 · Supervisor, 75 % feedback

Asked for	*Use Levenshtein distance or a fuzzy-matching threshold (e.g. ≥ 90 % similarity) in the evaluation script.*
Status	**Done**
What was done	RapidFuzz ratio ≥ 90.
See also	S4

M3 · Supervisor, 75 % feedback

Asked for	*Document the use of bitsandbytes and test whether 4-bit quantization reduces latency and memory on the RTX 3050 without losing accuracy.*
Status	**Done**
What was done	All four precisions were benchmarked with every invoice on one model (3 runs each).
Result	4-bit: 1102 MB vs fp32 1645 MB (33 % less memory); LLM time 0.115 vs 0.115 s (no speed-up); F1 0.9406 vs 0.9406. fp16: 0.079 s, 1049 MB. 8-bit: 0.258 s.
Documented in	Architecture_and_Benchmark_Report.pdf, section 6.8
See also	S3

M4 · Supervisor, 75 % feedback

Asked for	*Use the 600-document results to statistically finalise the routing thresholds (exact token count and OCR confidence).*
Status	**Done**
What was done	Every invoice is run through both routes (all-4-bit and all-fp32); a grid of thresholds (tokens 50–800, confidence 70–95) is simulated and the policy with the largest 4-bit share whose accuracy stays within 0.5 percentage points of all-fp32 is chosen, then validated with 3 fresh runs.
Result	Chosen: 525 tokens, confidence 70 → 99.8 % of invoices on the 4-bit model; B-tuned F1 0.9406, 126.1 s.
Documented in	outputs/2026-10-18_23-52-41_full_benchmark/routing_analysis.md

evaluation/routing_analysis.py · lines 14–14

```
14  TOKEN_GRID = list(range(50, 801, 25))
```

evaluation/routing_analysis.py · lines 75–77

```
75      floor = all_full["field_accuracy"] - tolerance
76      admissible = grid[grid["field_accuracy"] >= floor]
77      chosen = admissible.sort_values(["fast_share", "field_accuracy"], ascending=[False, False]).iloc[0]
```

E1 · Supervisor, e-mails

Asked for	*Three runs per configuration.*
Status	**Done**
What was done	Every configuration has 3 runs; tables give mean ± standard deviation.
See also	C3

E2 · Supervisor, e-mails

Asked for	*Report cost in € per 100 documents.*
Status	**Done**
What was done	Cost = wall time × €0.35 per GPU-hour, per document and per 100 documents.
Result	A €0.0110, B €0.0019 per 100 invoices.

pipeline/metrics.py · lines 123–124

```
123     total_hours = wall_time / 3600
124     total_cost = total_hours * GPU_HOURLY_COST_EUR
```

pipeline/metrics.py · lines 144–144

```
144         "cost_per_100_docs_eur": round(total_cost / docs * 100, 4) if docs else 0.0,
```

E3 · Supervisor, e-mails

Asked for	*Document the hardware (RTX 3050).*
Status	**Done**
What was done	GPU, CPU, RAM, OS, Python, PyTorch, CUDA and library versions are written into every run's summary.json and run_info.json.
Result	NVIDIA GeForce RTX 3050 Laptop GPU 4 GB, 12 logical cores, 15.7 GB RAM, Windows 11, Python 3.12.2, PyTorch 2.5.1 + CUDA 12.1.

evaluation/run_experiments.py · lines 54–71

```
54 def hardware_info() -> dict:
55     import psutil
56     import torch
57 
58     info = {
59         "python": platform.python_version(),
60         "platform": platform.platform(),
61         "cpu": platform.processor(),
62         "cpu_count": psutil.cpu_count(logical=True),
63         "ram_total_gb": round(psutil.virtual_memory().total / 2**30, 1),
64         "torch": torch.__version__,
65         "cuda": torch.version.cuda,
66     }
67     if torch.cuda.is_available():
68         properties = torch.cuda.get_device_properties(0)
69         info["gpu"] = properties.name
70         info["gpu_vram_gb"] = round(properties.total_memory / 2**30, 1)
71     return info
```

3. What the author still has to write

These parts are the thesis text itself. The material each one needs is ready:

Thesis part	Requirement	Material ready
Analytical part (A-3, A-4): literature review, existing solutions and their weaknesses, technology choices	C4, T1, T2	Literature_Sources.pdf (verified sources with summaries); Thesis_Outline.pdf chapter 1; measured comparisons P5, M3, S2, P1
Why OCR is still needed next to VLMs	C1	VLM measurement on the same GPU (C1); sources in Literature_Sources.pdf, section 2
Chapter order: analysis → design → dataset and method → results	C5	Thesis_Outline.pdf; Architecture_and_Benchmark_Report.pdf follows the same order
Discussion and future work	T6	Architecture_and_Benchmark_Report.pdf, sections 6–8 (trade-offs and measured improvement options)

4. How to check it yourself

What	Command
Unit tests	`.venv\Scripts\python -m pytest tests -q`
Main benchmark (all configurations, 3 runs)	`.venv\Scripts\python -m evaluation.run_experiments --configs all --runs 3`
Latency with invoices arriving one at a time	`.venv\Scripts\python -m evaluation.latency_experiment --docs 60 --runs 3`
Tesseract vs EasyOCR	`.venv\Scripts\python -m evaluation.ocr_comparison`
VLM baseline	`.venv\Scripts\python -m evaluation.vlm_baseline`
Routing thresholds from data	`.venv\Scripts\python -m evaluation.routing_analysis --results`